Sprint 3: Table Manipulation

LEVEL 1

- **EXERCISE 1**

```sql
-- SPRINT 3
-- LEVEL 1
-- Exercise 1
-- We create the table Credit Card
CREATE TABLE IF NOT EXISTS credit_card (
    id VARCHAR(255) PRIMARY KEY,
    iban VARCHAR(255),
    pan VARCHAR(255),
    pin VARCHAR(255),
    cvv VARCHAR(255),
    expiring_date VARCHAR(255));

DESCRIBE credit card;
```

100% 33:39

Action Output

	Time	Action	Response
1	18:03:27	CREATE TABLE IF NOT EXISTS credit_card (...	0 row(s) affected

- We created the **table "*credit_card*"** based on the column information given in the file *"dades_introduir_credit.sql"* and insert the data of the table from the same file.

```sql
DESCRIBE credit_card;

-- The fk creation is not possible yet due to some i
-- To detect these missing values we left join both
```

100% 22:41

Result Grid Filter Rows: Search Export:

Field	Type	Null	Key	Default	Extra
id	varchar(255)	NO	PRI	NULL	
iban	varchar(255)	YES		NULL	
pan	varchar(255)	YES		NULL	
pin	varchar(255)	YES		NULL	
cvv	varchar(255)	YES		NULL	
expiring_date	varchar(255)	YES		NULL	

Result 3

Action Output

	Time	Action	Response
49...	18:03:52	INSERT INTO credit_card (id, iban, pan, pin...	1 row(s) affected
49...	18:03:52	INSERT INTO credit_card (id, iban, pan, pin...	1 row(s) affected
49...	18:03:52	INSERT INTO credit_card (id, iban, pan, pin...	1 row(s) affected
49...	18:03:52	INSERT INTO credit_card (id, iban, pan, pin...	1 row(s) affected
49...	18:03:52	INSERT INTO credit_card (id, iban, pan, pin...	1 row(s) affected
49...	18:03:52	INSERT INTO credit_card (id, iban, pan, pin...	1 row(s) affected
49...	18:03:52	INSERT INTO credit_card (id, iban, pan, pin...	1 row(s) affected
50...	18:03:52	INSERT INTO credit_card (id, iban, pan, pin...	1 row(s) affected
50...	18:03:52	INSERT INTO credit_card (id, iban, pan, pin...	1 row(s) affected
50...	18:04:12	DESCRIBE credit_card	6 row(s) returned

- The table has six columns from which "credit_card.id" is the Primary Key.

```sql
SELECT t.credit_card_id
FROM transaction t
LEFT JOIN credit_card cc ON t.credit_card_id = cc.id
WHERE cc.id IS NULL;
```

0% 21:48

Result Grid Filter Rows: Search Export:

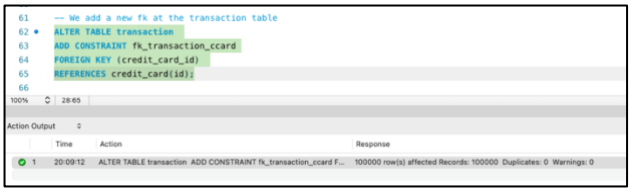
credit_card_id

Result 8

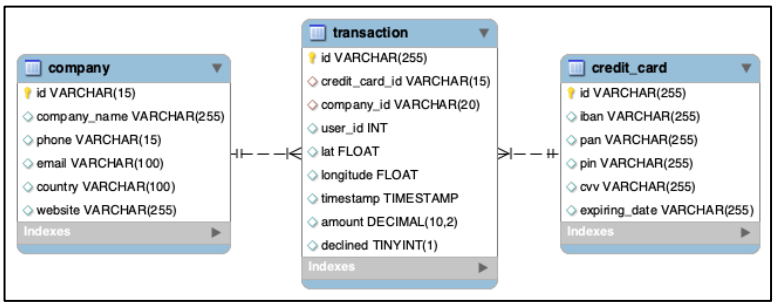
ion Output

	Time	Action	Response
1	18:20:03	SELECT t.credit_card_id FROM transaction t...	0 row(s) returned

- To set "credit_card.id" as a foreign key in the ***transaction*** table ("transaction.credit_card_id") first we look for null id values in ***credit_card*** that are not null in transaction.
We can see that all credit card ids are present in the table ***transaction.***



- Now we can set "credit_card_id" as a foreign key in ***transaction*** table, having ***credit_card*** as parent table.



The ER diagram shows the relationship **one to many (1:N)** between the ***credit_card*** and the ***transaction*** tables. This means that one credit card may have multiple transactions.

The table ***company*** has a similar relationship with ***transaction***: one company may have multiple transactions registered.

- **EXERCISE 2**

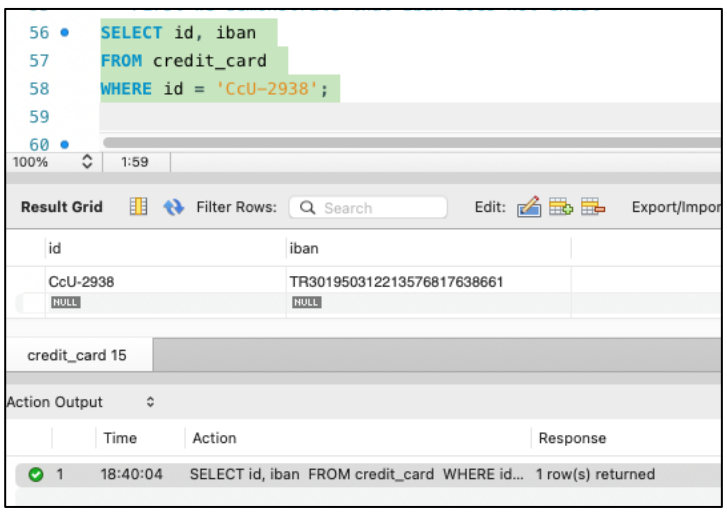


Fig. 1 Testing the existence of Iban for the card id "CcU-2938".

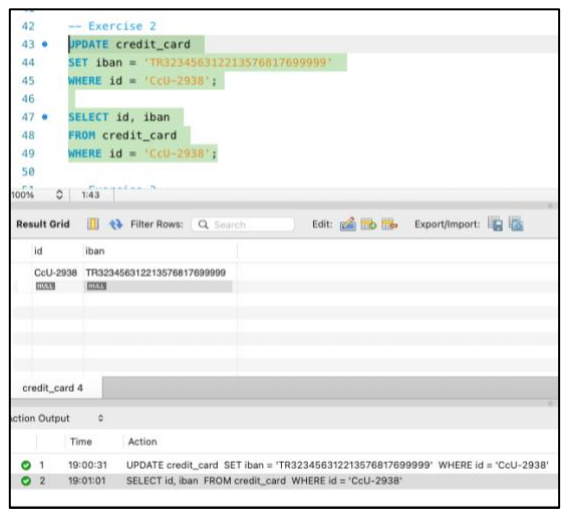


Fig. 2 Iban modification for specific credit card Id.

To modify the account number (Iban) value for the credit card id "CcU-2938" first we observe the current number (Fig. 1). Then, we use the UPDATE statement to set the new account number for the credit card id "CcU-2938" in the ***credit_card*** table and finally we confirm the modification (Fig. 2).

- **EXERCISE 3**

```
-- Exercise 3
INSERT INTO transaction (id, credit_card_id, company_id, user_id, lat, longitude, amount, declined)
VALUES ('108B1D1D-5B23-A76C-55EF-C568E49A99DD', 'CcU-9999', 'b-9999', '9999', 829.999, -117.999, 111.11, 0);
```

	Time	Action	Response
1	20:09:12	ALTER TABLE transaction ADD CONSTRAINT fk_transaction_ccard F...	100000 row(s) affected Records: 100000 Duplicates: 0 Warnings: 0
2	20:11:25	SELECT iban FROM credit_card WHERE iban = 'TR323456312213576...	0 row(s) returned
3	20:52:49	INSERT INTO transaction (id, credit_card_id, company_id, user_id, lat...	Error Code: 1452. Cannot add or update a child row: a foreign key constraint fails ("transactions", "transaction", CONSTRAINT "trans

When we try to insert a new value the error 1452 appears.

```
SELECT c.id
FROM company c
WHERE c.id = 'b-9999';
```

id
NULL

company 9

	Time	Action
1	21:03:37	SELECT c.id FROM company c WHERE c.id = 'b-9999'

Given that the company id does not exist in the parent table ***company***, is not possible to insert a value for this company in the child table ***transaction***.

```
-- We insert the compny id in the company table
INSERT INTO company (id, company_name)
VALUES ('b-9999', 'IT_academy');
```

	Time	Action	Response
1	21:10:30	INSERT INTO company (id, company_name) VALUES ('b-9999', 'IT_ac...	1 row(s) affected

To solve this problem, we first add the new id at the ***company*** table

```
-- We check the existence of the credit card id at the parent table credit_card
SELECT cc.id
FROM credit_card cc
WHERE cc.id = 'CcU-9999';
```

id
NULL

credit_card 11

	Time	Action	Response
1	21:14:27	SELECT cc.id FROM credit_card cc WHERE cc.id = 'CcU-9999'	0 row(s) returned

We also have a similar issue in the ***credit_card*** table. The id value that we want to insert does not exist in the parent table

```
-- >>>>>> Problems with credit_card table
-- We check the existence of the credit card id at the parent table credit_card
SELECT cc.id
FROM credit_card cc
WHERE cc.id = 'CcU-9999';

-- We insert the credit card id in the credit_card table
INSERT INTO credit_card (id)
VALUES ('CcU-9999');

-- we confirm the existence of cc id
SELECT cc.id
FROM credit_card cc
WHERE cc.id = 'CcU-9999';
```

100% 1:97

Result Grid Filter Rows: Search Edit: Export/Import:

id
CcU-9999
NULL

credit_card 12

Action Output

	Time	Action	Response
1	21:14:27	SELECT cc.id FROM credit_card cc WHERE cc.id = 'CcU-9999'	0 row(s) returned
2	21:16:30	INSERT INTO credit_card (id) VALUES ('CcU-9999')	1 row(s) affected
3	21:17:15	SELECT cc.id FROM credit_card cc WHERE cc.id = 'CcU-9999'	1 row(s) returned

To solve this issue, we followed the same approach than above.
We insert the new id value at the ***credit_card*** parent table and test its existence.

```
-- Exercise 3
INSERT INTO transaction (id, credit_card_id, company_id, user_id, lat, longitude, amount, declined)
VALUES ('108B1D1D-5B23-A76C-55EF-C568E49A99DD', 'CcU-9999', 'b-9999', '9999', 829.999, -117.999, 111.11, 0);
```

100% 14:55

Action Output

	Time	Action	Response
1	19:06:09	INSERT INTO transaction (id, credit_card_id, company_id, user_id, lat, longitude, amount, declined) VALUES ('108B1D1D-5B23-A76C-55EF-C568E49A9...	1 row(s) affected

Now, we can successfully add the new values in ***transaction*** table

- **EXERCISE 4**

```
-- Exercise 4
ALTER TABLE credit_card
DROP COLUMN pan;
DESCRIBE credit_card;
```

100% 17:57

Result Grid Filter Rows: Search Export:

Field	Type	Null	Key	Default	Extra
id	varchar(15)	NO	PRI	NULL	
iban	varchar(50)	YES		NULL	
pin	varchar(4)	YES		NULL	
cvv	varchar(4)	YES		NULL	
expiring_date	varchar(10)	YES		NULL	

Result 6

Action Output

	Time	Action
1	19:09:33	ALTER TABLE credit_card DROP COLUMN pan
2	19:09:48	DESCRIBE credit_card

Using ALTER statement we remove the column "pan" from the ***credit_card*** table

LEVEL 2

- **EXERCISE 1**

  To remove a record from the data base, first we use the id to confirm its existence at the ***transaction*** table (Fig. 3), then we remove the record using the id as a condition and confirm that it does not exist anymore (Fig. 4).

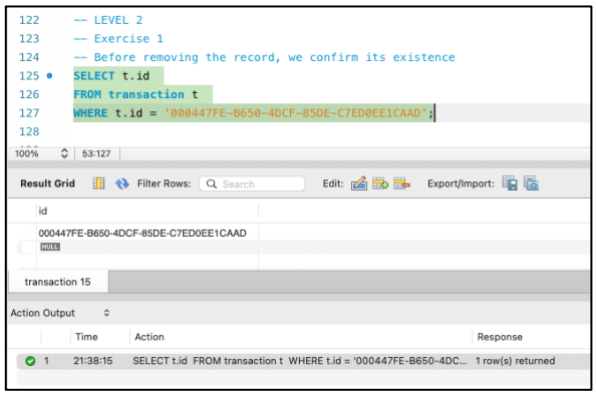


Fig. 3 We test the existence of the record using the id

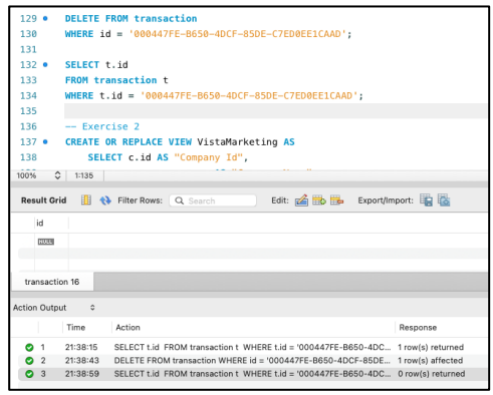
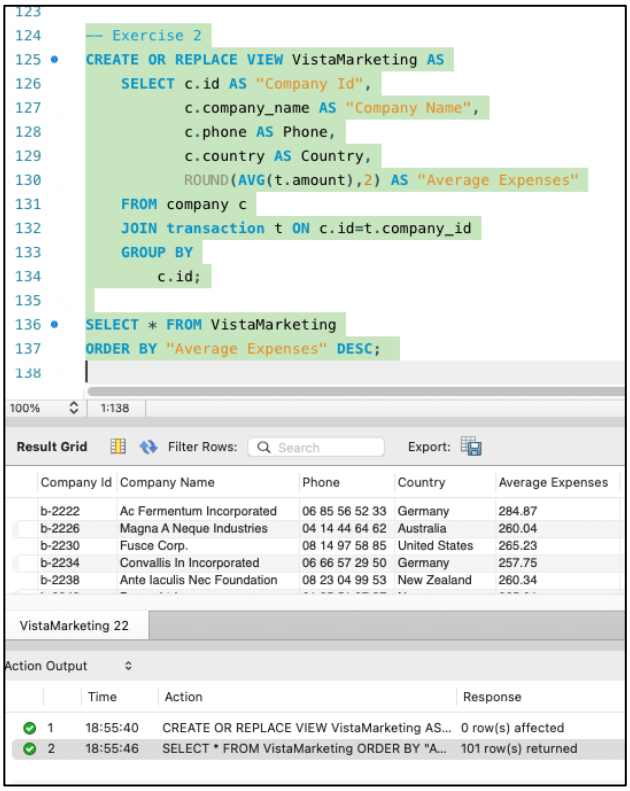


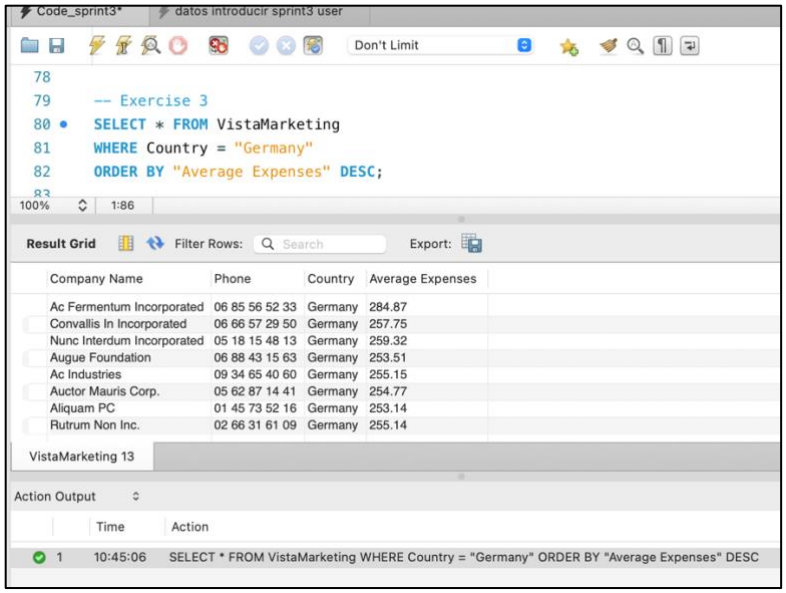
Fig. 4 We remove the record and re confirm the action

- **EXERCISE 2**

  We create a view using the needed information



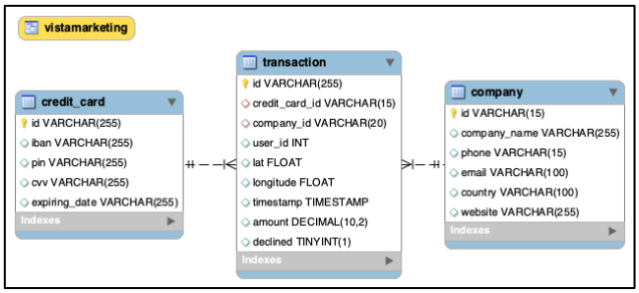
- **EXERCISE 3**

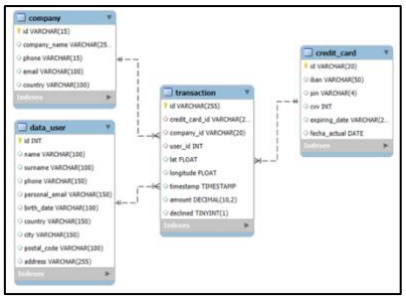


LEVEL 3

- **EXERCISE 1**

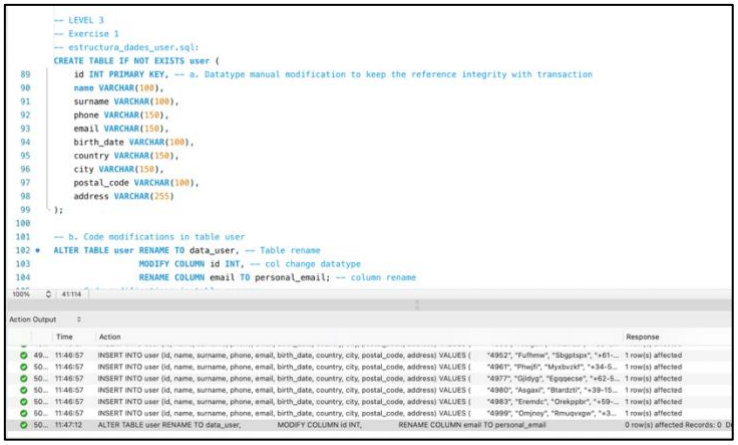
The ER diagrams below show the current and new design of the ER diagram.





To obtain the entire code based on the new ER diagram we made the following modifications:

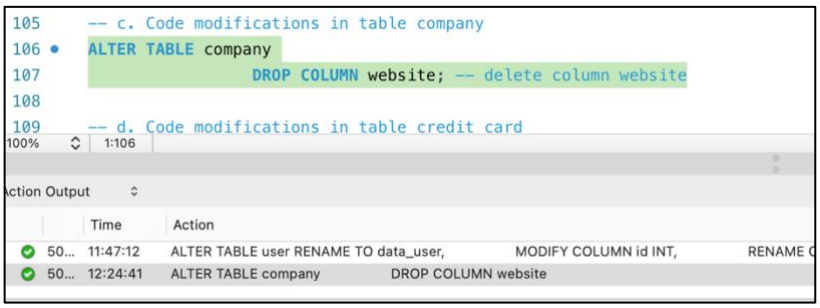
a. We create a new table ***user*** using estructura_dades_user.sql script
b. Insert the records



```
-- LEVEL 3
-- Exercise 1
-- estructura_dades_user.sql:
CREATE TABLE IF NOT EXISTS user (
    id INT PRIMARY KEY, -- a. Datatype manual modification to keep the reference integrity with transaction
    name VARCHAR(100),
    surname VARCHAR(100),
    phone VARCHAR(150),
    email VARCHAR(150),
    birth_date VARCHAR(100),
    country VARCHAR(150),
    city VARCHAR(150),
    postal_code VARCHAR(100),
    address VARCHAR(255)
);

-- b. Code modifications in table user
ALTER TABLE user RENAME TO data_user, -- Table rename
                 MODIFY COLUMN id INT, -- col change datatype
                 RENAME COLUMN email TO personal_email; -- column rename
```

c. We change the name of the **table** *user* to *data_user*, the **datatype** from "CHAR(10)" to "INT" at the column *id* and rename **column** *email* to *personal_email* in the same table.



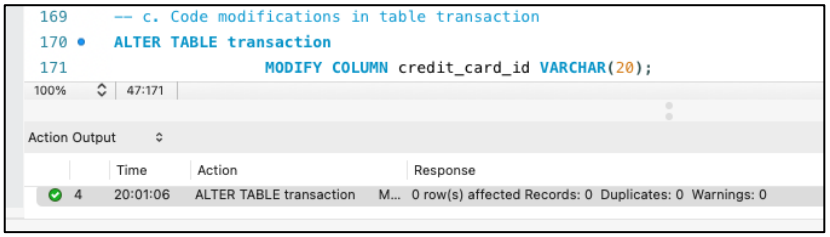
```
-- c. Code modifications in table company
ALTER TABLE company
                 DROP COLUMN website; -- delete column website

-- d. Code modifications in table credit card
```

Action Output

	Time	Action
50...	11:47:12	ALTER TABLE user RENAME TO data_user, MODIFY COLUMN id INT, RENAME C
50...	12:24:41	ALTER TABLE company DROP COLUMN website

d. We remove **column** *website* from **table** *company*

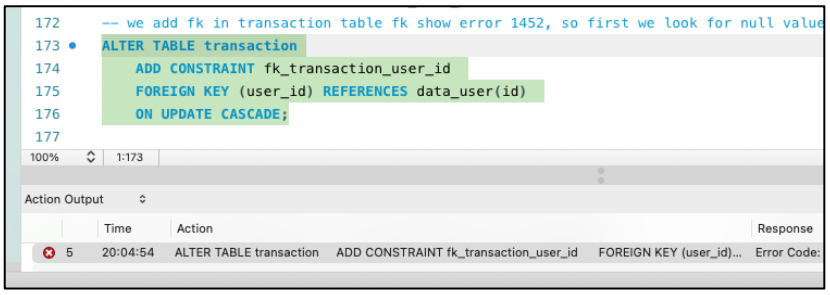


```
-- c. Code modifications in table transaction
ALTER TABLE transaction
                 MODIFY COLUMN credit_card_id VARCHAR(20);
```

Action Output

	Time	Action	Response
4	20:01:06	ALTER TABLE transaction M...	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0

e. We modify datatype size of credit card id in ***transaction*** table



```
-- we add fk in transaction table fk show error 1452, so first we look for null value
ALTER TABLE transaction
    ADD CONSTRAINT fk_transaction_user_id
    FOREIGN KEY (user_id) REFERENCES data_user(id)
    ON UPDATE CASCADE;
```

Action Output

	Time	Action	Response
5	20:04:54	ALTER TABLE transaction ADD CONSTRAINT fk_transaction_user_id FOREIGN KEY (user_id)...	Error Code:

f. We try to add a foreign key to the same table, but we get error 1452 because of a referential integrity violation

```
-- ADD fk show error 1452, so first we look for null values
SELECT DISTINCT t.user_id
FROM transaction t
LEFT JOIN data_user du ON t.user_id = du.id
WHERE du.id IS NULL;
```

100% 21:176

Result Grid Filter Rows: Search Export:

user_id
9999

Result 24

ction Output

	Time	A...	Response
1	19:39:18	S...	1 row(s) returned

g. We check the existence of user ids that are not present in ***data_users*** table and we detect id 9999

```
SELECT *
FROM data_user
WHERE id = 9999;

INSERT INTO data_user (id)
VALUES ('9999');

SELECT *
FROM data_user
WHERE id = 9999;
```

100% 17:187

Result Grid Filter Rows: Search Edit: Export/Import:

id	name	surname	phone	personal_email	birth_da...	country	city	postal_code	address
9999	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

data_user 26

Action Output

	Time	A...	Response
1	19:39:18	S...	1 row(s) returned
2	19:41:07	S...	0 row(s) returned
3	19:41:35	IN...	1 row(s) affected
4	19:41:43	S...	1 row(s) returned

h. We confirm first that there is not user id 9999, then we insert the id value and recheck its existence in ***data_users*** table

```
;
ALTER TABLE transaction
    ADD CONSTRAINT fk_transaction_ccard
    FOREIGN KEY (credit_card_id) REFERENCES credit_card(id)
    ON UPDATE CASCADE;
```

100% 21:209

Action Output

	Time	A...	Response
1	19:45:51	A...	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0
2	19:46:02	A...	5001 row(s) affected Records: 5001 Duplicates: 0 Warnings: 0
3	19:47:13	A...	100000 row(s) affected Records: 100000 Duplicates: 0 Warnings: 0

i. We add a new foreign key to ***transaction*** table with ***credit_card*** as parent table

```
ALTER TABLE transaction DROP CONSTRAINT fk_transaction_ccard;
-- d. Code modifications in table credit_card
ALTER TABLE credit_card
                ADD fecha_actual DATE, -- add new column + datatype
                MODIFY COLUMN id VARCHAR(20),
                MODIFY COLUMN iban VARCHAR(50),
                MODIFY COLUMN pin VARCHAR(4),
                MODIFY COLUMN cvv INT,
                MODIFY COLUMN expiring_date VARCHAR(20)
                ;
-- e. NEW Code modifications in table transaction
ALTER TABLE transaction
    ADD CONSTRAINT fk_transaction_ccard
    FOREIGN KEY (credit_card_id) REFERENCES credit_card(id)
    ON UPDATE CASCADE;
```

100% 21:214

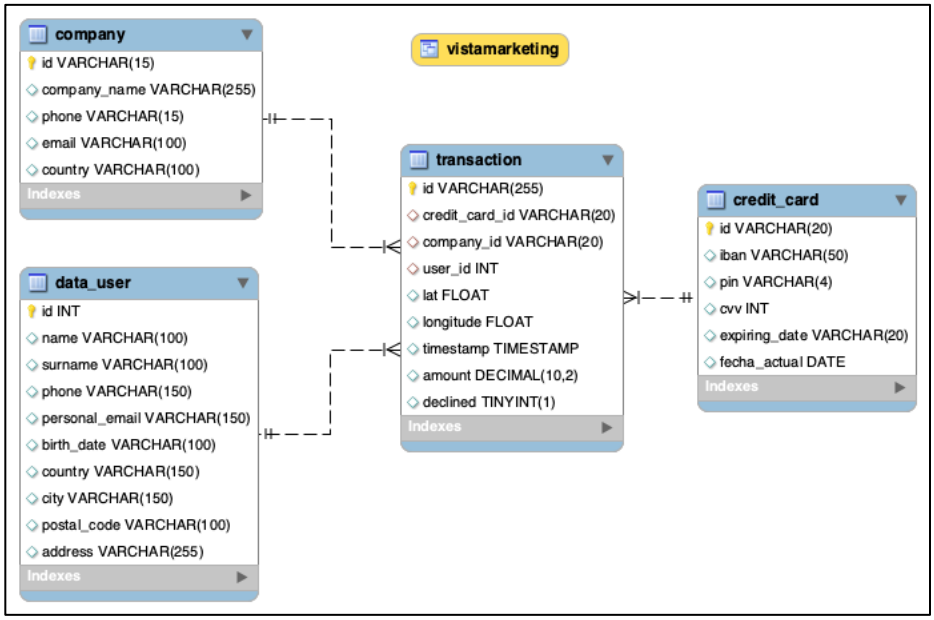
Action Output

	Time	Action	Response
7	20:15:06	ALTER TABLE transaction ADD CONSTRAINT fk_transaction_ccard FOREIGN KEY (credit_card_id...	100000 row(s) affected Rec

j. Before modifying ***credit_card(id)*** size, we need to remove this column as a foreign key in ***transaction*** table

k. Then we modify ***credit_card*** columns changing datatype sizes and add a new **column** named *fecha_actual* with DATE datatype

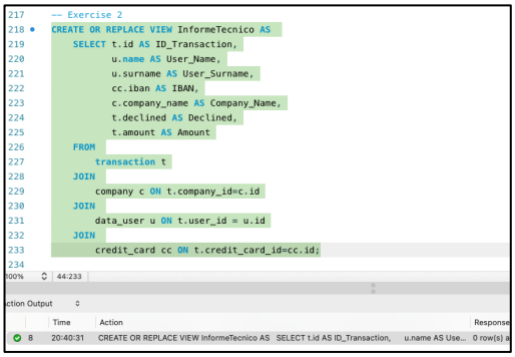
l. Finally, we set *credit_card_id* as a foreign key in ***transaction*** table



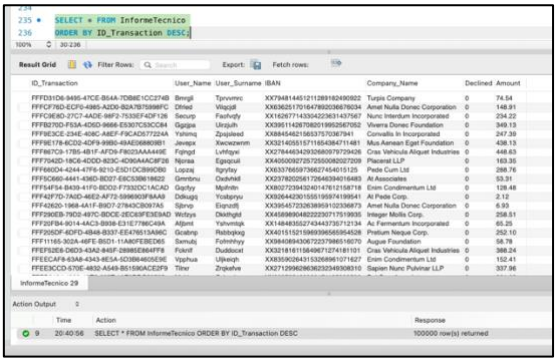
Final ER diagram after all code modifications to match the desired ER. The relationship between the ***data_user*** is also 1:N with ***transaction*** table, one user can have many different transactions.

- **EXERCISE 2**

  To create the view "*InformeTecnico*" we joined the tables ***data_user, company and credit_card*** with ***transaction*** including extra relevant information.



```
-- Exercise 2
CREATE OR REPLACE VIEW InformeTecnico AS
    SELECT t.id AS ID_Transaction,
           u.name AS User_Name,
           u.surname AS User_Surname,
           cc.iban AS IBAN,
           c.company_name AS Company_Name,
           t.declined AS Declined,
           t.amount AS Amount
    FROM
        transaction t
    JOIN
        company c ON t.company_id=c.id
    JOIN
        data_user u ON t.user_id = u.id
    JOIN
        credit_card cc ON t.credit_card_id=cc.id;
```



```
SELECT * FROM InformeTecnico
ORDER BY ID_Transaction DESC;
```

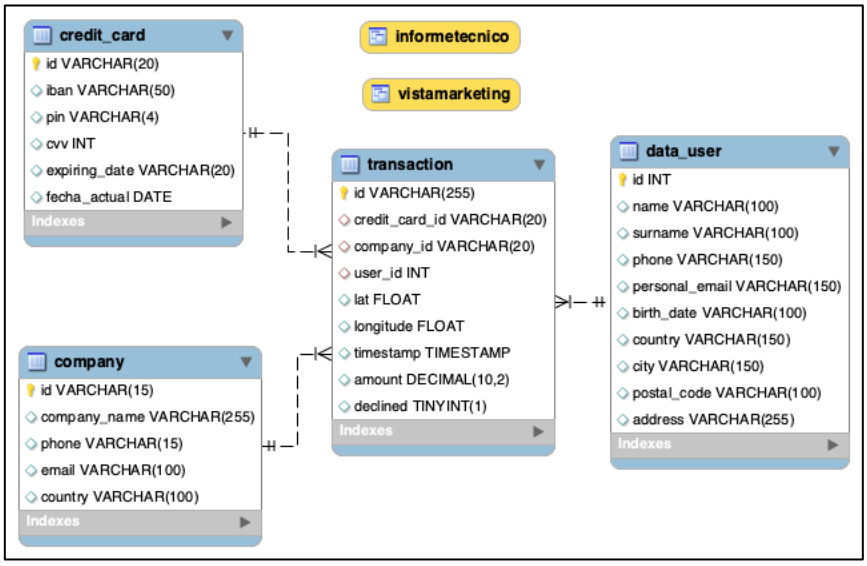


Fig. 2 Final ER diagram for the transactions database